Module: ITNPBD2 Autumn 2025
Assessment: Main assignment
Due Date/Time: 14/11/2025
AIAS Levels Allowed: 2

	Please tick the boxes/include appropriate information below
Student ID Number	3523330
Word Count (penalties apply for exceeding the stated limit)	20pgs
I have read and understand the severity of academic misconduct – see link below	☒
I give consent for my work to be used as an exemplar to future students.	☒
I have checked my submitted document to ensure it complies with module requirements.	☒
Link to version-controlled file (i.e on OneDrive, Google Docs, Github, or other) which contain evidence of the process I undertook to complete this assignment. Information on how to create a Microsoft 365 OneDrive folder is available HERE. *Please see notes below	R&MD Project
I understand that if there is a concern about potential academic misconduct, including inappropriate use of AI tools, then I could be asked to provide evidence of my drafting process during an academic integrity meeting if I have not done so using the link above. Not providing evidence of my drafting process could prejudice the outcome of academic misconduct cases.	☒
Tailored feedback. If you would like tailored feedback on a specific aspect (or aspects) of your work (e.g., referencing, writing style, grammar), then please give details here.	I would like detail feedback on everything that I did wrong and could have been better.
If you used AI at (or below) the level allowed, please explain briefly which AI, how you used it, and for what purpose.	**I used chat AI to brainstorm the idea**

**This may include (but is not limited to) drafts, versions of the finished document, notes, references, AI output, and AI prompts. These materials are not marked or graded, but they are simply a way to demonstrate how your work was created and to confirm that any AI use in your final submission is within the permitted AIAS scale for your assessment. Providing this helps safeguard you, showing your authentic process, and protecting you should any academic integrity questions arise.*

https://www.stir.ac.uk/about/professional-services/student-academic-and-corporate-services/academic-registry/policy-and-procedure/

For more information, please see the Coversheet FAQ.

DATA ANALYSIS REPORT

Identifying Hidden Gem Products for JC Penney

Using Structured Data, JSON Metadata & NLP Sentiment Analysis

Table of Contents

1. INTRODUCTION

JC Penney have thousands of diverse products across different categories, yet a small fraction of products do not receive significant customer attention. Not because they are poor in quality or are not in good quality. The reason is Customers haven't discovered them yet.

This report analyses whole catalog of JC Penney's products, customer reviews, and products metadata to answer a single strategic question:

To answer this question, we combined:

- Product information
- Customer Review scores
- Customer sentiments from review
- prices and discount information

We developed a Hidden Gem Score, a single Measure that identifies customers love for products despite there low visibility.

Our findings reveal clear opportunities for improved merchandising, prioritization, and targeted marketing.

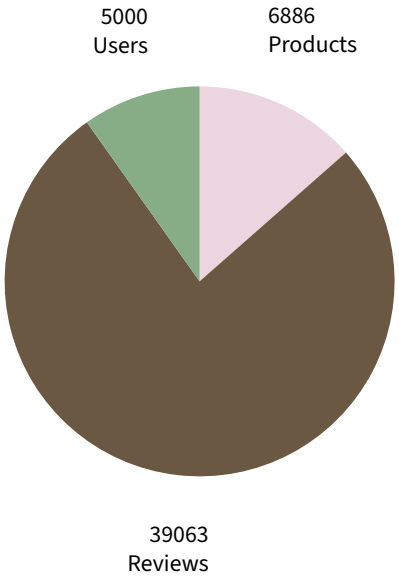
Which products deliver exceptional customer satisfaction but remain underexposed and therefore represent missed revenue opportunities?

2. BODY

2.1 Data Overview

For this analysis we integrated five JC Penney data sources and they provided us with following information:

- **Product**
  Included SKU, name, description, prices, brand, category, and image links of the products
- **User**
  Included the name, state and date of birth of user.
- **Reviews**
  Included User name and his comment about the product as well as a numeric score about the product.



This combination of data allowed us to evaluate each product from three angles:

1. **Customer satisfaction** (rating)
2. **Customer emotion** (sentiment of review text)
3. **Customer engagement** (number of reviews)

2.2 Methods

The analysis of dataset is done in python but the approach can be summarized simply:

1. **Clean and Unify all the information needed**
   - Made SKU standard product identifiers.
   - Removed duplicate and useless records.
   - Merged brand, category, and image metadata into the main product table.

2. **Evaluate customer opinion**
   - Every review was analyzed on already provided rating score.
   - A separate sentiment score was also calculated. sentiment score is the measure of positive or negative emotion in written review.

3. **Build product level performance metrics**

   Following was calculated for each product:
   - Average rating
   - Average Sentiment
   - Total number of reviews
   - Price and discount information
   - Brand and category

4. **Hidden Gem Score**

   We Used following formula to calculate HGS.

$$\text{Hidden Gem Score} = \frac{\text{Rating} \times \text{Sentiment}}{\log(\text{Number of Reviews} + 2)}$$

This score highlights:

- Products with strong positive reviews.
- But have low review count
- Therefore, they are excellent but under discovered.

2.3 Results

Through our analysis we identified several products with outstanding customer satisfaction and highly positive sentiments but they had fewer than 5 total reviews. This is highly useful because these products are low risk. why? because they have already proven customer satisfaction but haven't reached larger audience. This means that we have an opportunity to promote these products and get profit.

To put the Hidden Gem Score on a scale of real values, we first needed to calculate its theoretical maximum, using best-case inputs: a perfect 5-star rating, fully positive sentiment, and the minimum of one review.

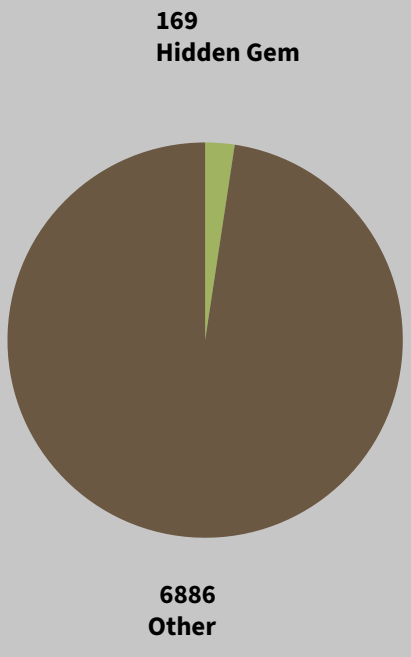
This gave us a highest possible score of 4.55. We sought to identify only truly outstanding products, so we set the threshold at half this value, or 2.28.

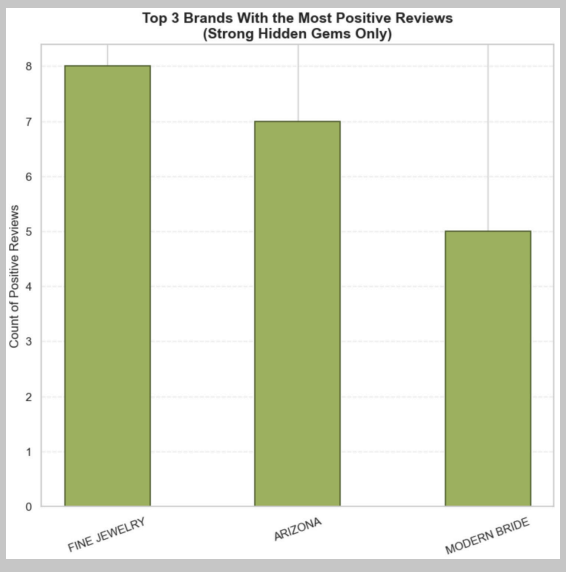
Any item scoring above 2.28 is considered a strong hidden gem, as it reaches an unusually high customer satisfaction given how few reviews it has received.

By applying a threshold value for the Hidden Gem Score of 2.28, which is half of the theoretical max score of 4.55, we isolated 169 products that outperform the rest of the catalog in terms of customer satisfaction while receiving minimal visibility.

These items represent only 2.8% of the full product range, which means that hidden gems are rare but high-impact when found.

The results also show that certain brands notably Fine jewelry show high sentiment and rating scores. Which implies that brands like these deserve priority placement, potential expansion and maybe seasonal promotion campaigns as well.





CONCLUSION & RECOMMENDATIONS

The analysis surfaced a small number of high-value items that maintain exceptional customer satisfaction despite having less than five reviews. These strong hidden gems demonstrate excellent ratings and highly positive sentiment, meaning they are low-risk opportunities with clear potential for increased revenue. Using a theoretical maximum Hidden Gem Score of 4.55 and selecting a threshold of 2.28, we isolated 169 products-just 2.8% of the catalog-that outperform expectations but lack visibility.

Recommendations:

1. Increase Visibility of Hidden Gem Products

These are items that have already shown a strong appeal to customers. By growing their visibility with homepage banners, personalized recommendations, top-of-category placement, or targeted email slots, quick wins with minimal risk can be captured.

2. Favor High-Performing Brands

Certain brands are over performing with very positive sentiment consistently in Fine Jewelry. These brands deserve to:

- priority placement in merchandising layouts
- expanded product inventory for top performers
- dedicated "Most Loved" or "Hidden Favorites" campaigns
- targeted promotional opportunities in peak seasons.

3. Integrate Hidden Gem Scoring into Marketing Workflow

The Hidden Gem Score could become a continuous internal signal to underline emerging opportunities. Integrating it into merchandising dashboards would automatically show teams:

under-discovered products worth promoting Products that elicit a strong customer response but have low visibility potential best-sellers before they trend Products that deserve better imagery, descriptions, or feature placement.

3523330_BD2_Assignment

November 19, 2025

1 JC Penney Hidden Gems Analysis

1.0.1 Combining Product Metadata, Customer Reviews & Sentiment Analysis

1.0.2 Summary

This notebook identifies "Hidden Gem" products at JC Penney. The items that customers love but do not receive enough visibility.
Using data from product tables, user reviews, and JSON metadata, we merged structured and unstructured information to measure:

- **Customer satisfaction** (rating)
- **Customer emotion** (sentiment score)
- **Customer engagement** (number of reviews)
- **Product visibility indicators** (category, brand, images)

The goal is simple: highlight products that deserve promotion.

1.0.3 1. Setup

We import essential analysis libraries.

[98]:
```
import os
import re
import json
import math
import numpy as np
import pandas as pd
from pathlib import Path

import matplotlib.pyplot as plt
import seaborn as sns

import nltk
from nltk.sentiment import SentimentIntensityAnalyzer

print(" Setup ready.")
```

```
 Setup ready.
```

1.0.4 2. Loading the Data

We load the CSV and JSON files and prepare them for analysis.

[99]:
```
# file paths
base = Path("JCPenneyFiles")
products = pd.read_csv(base / "products.csv", encoding="utf-8",␣
 ↪on_bad_lines="skip", dtype=str)
reviews  = pd.read_csv(base / "reviews.csv",  encoding="utf-8",␣
 ↪on_bad_lines="skip", dtype=str)
users    = pd.read_csv(base / "users.csv",    encoding="utf-8",␣
 ↪on_bad_lines="skip", dtype=str)

# clean column names
def clean(df):
    df.columns = df.columns.str.strip().str.lower().str.replace(r"\s+", "_",␣
 ↪regex=True)
    return df

products = clean(products)
reviews  = clean(reviews)
users    = clean(users)

# convert numeric columns
num_cols = ["price", "av_score", "score"]
for col in num_cols:
    for df in (products, reviews):
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")
```

[100]:
```
from IPython.display import display

#print("Products:")
#display(products.head())

#print("Reviews:")
#display(reviews.head())

#print("Users:")
#display(users.head())
```

[101]:
```
import json

def load_json_lines(path):
    with open(path, "r", encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]

# Load files
```

```
jcp_products_json = load_json_lines("JCPenneyFiles/jcpenney_products.json")
jcp_reviewers_json = load_json_lines("JCPenneyFiles/jcpenney_reviewers.json")
```

1.0.5 3. Data Cleaning

We prepare the data so it can be safely analyzed.
Key steps:

- Remove duplicates
- Standardize columns
- Convert price fields to numeric
- Use `sku` as the single product identifier
- Extract useful metadata from JSON
- Keep users, reviews, and products separate (normalization)

[102]:
```
import warnings
warnings.filterwarnings("ignore")
```

[103]:
```
# create a unique product ID so we can isolate the single best-rated version␣
↪(max avg score)

unique_products = (
    products
    .drop_duplicates(subset=["sku", "name", "description", "price"])
    .reset_index(drop=True)
)
```

[104]:
```
#Now link every old uniq_id to the matching real product so we can use it as␣
↪unique key:
mapping = products[["uniq_id", "sku"]].drop_duplicates()
```

[105]:
```
#Removed unique key and av score because we want SKU as unique key
products_clean = unique_products.drop(columns=["uniq_id", "av_score"],␣
↪errors="ignore")
```

[106]:
```
# reviews was linked with product through uniq_id, we changed it to SKU as well
reviews_clean = reviews.merge(mapping, on="uniq_id", how="left")
reviews_clean = reviews_clean.drop(columns=["uniq_id"])
```

[107]:
```
#Saved the clean files
products_clean.to_csv("products_clean.csv", index=False)
reviews_clean.to_csv("reviews_clean.csv", index=False)
mapping.to_csv("uniqid_sku_mapping.csv", index=False)
```

[108]:
```
# To verify if it worked
#print("Products:")
#display(products_clean.head())
```

```
#print("Reviews:")
#display(reviews_clean.head())
```

1.0.6 4. Merging Metadata

The JSON file contains additional product information (brand, category, image URL).
We merge these fields to enrich the cleaned product table.

[109]:
```
import pandas as pd

# load products into a flat dataframe
products_df = pd.DataFrame(jcp_products_json)

# drop nested relationship columns to keep products table normalized
if "Reviews" in products_df.columns:
    products_df = products_df.drop(columns=["Reviews"], errors="ignore")
if "Bought With" in products_df.columns:
    products_df = products_df.drop(columns=["Bought With"], errors="ignore")

print("Products DF:", products_df.shape)

reviews_rows = []

# flatten all review objects into one tidy reviews dataframe
for item in jcp_products_json:
    product_id = item.get("uniq_id")
    sku = item.get("sku")
    review_list = item.get("Reviews", [])

    for r in review_list:
        reviews_rows.append({
            "uniq_id": product_id,   # link review back to product
            "sku": sku,
            "username": r.get("User"),
            "review_text": r.get("Review"),
            "score": r.get("Score")
        })

reviews_from_json_df = pd.DataFrame(reviews_rows)
print("\nReviews From JSON (flattened):", reviews_from_json_df.shape)

# load reviewer metadata and normalize column names
reviewers_df = pd.DataFrame(jcp_reviewers_json)

reviewers_df = reviewers_df.rename(columns={
    "Username": "username",
    "DOB": "dob",
```

```
    "State": "state",
    "Reviewed": "reviewed_products"
})

print("\nReviewers DF:", reviewers_df.shape)
```

```
Products DF: (7982, 13)

Reviews From JSON (flattened): (39063, 5)

Reviewers DF: (5000, 4)
```

[110]:
```
# Save products from JSON
products_df.to_csv("products_from_json.csv", index=False)

# Save flattened reviews from JSON
reviews_from_json_df.to_csv("reviews_from_json.csv", index=False)

# Save reviewer info
reviewers_df.to_csv("reviewers_from_json.csv", index=False)

print("  All JSON-derived DataFrames saved as CSV files!")
```

```
 All JSON-derived DataFrames saved as CSV files!
```

[111]:
```
products_clean = pd.read_csv("products_clean.csv")
reviews_clean = pd.read_csv("reviews_clean.csv")
users_clean = pd.read_csv("JCPenneyFiles/users.csv")

products_json = pd.read_csv("products_from_json.csv")

# keep only fields pulled from JSON that enrich product metadata
json_product_cols = [
    "sku",
    "brand",
    "category",
    "category_tree",
    "list_price",
    "sale_price",
    "product_image_urls",
    "average_product_rating",
    "total_number_reviews"
]

# dedupe JSON products so each SKU has one row of extra info
products_json_small = products_json[json_product_cols].drop_duplicates("sku")

# merge cleaned products with JSON-enriched attributes
```

```
enhanced_products = products_clean.merge(
    products_json_small,
    on="sku",
    how="left"
)

print("Enhanced Products:", enhanced_products.shape)
#display(enhanced_products.head())
```

```
Enhanced Products: (6886, 12)
```

[112]:
```
enhanced_products.to_csv("enhanced_products.csv", index=False)
```

1.0.7 5. Sentiment Analysis

We score each review using VADER sentiment analysis, giving a value from -1 (negative) to +1 (positive).
This helps measure customer emotion beyond star ratings.

[113]:
```
# Apply sentiment analysis
reviews_clean["clean_review"] = reviews_clean["review"].apply(clean_text)
reviews_clean["sentiment"] = reviews_clean["clean_review"].apply(lambda x: sia.
 ↪polarity_scores(x)["compound"])

display(reviews_clean.head(1))
```

```
   username  score  \
0  fsdv4141      2

                                                                                ␣
 ↪                                                review  \
0  You never have to worry about the fit…Alfred Dunner clothing sizes are true␣
 ↪to size and fits perfectly. Great value for the money.

           sku  \
0  pp5006380337

                                                                                ␣
 ↪                                            clean_review  \
0  You never have to worry about the fit…Alfred Dunner clothing sizes are true␣
 ↪to size and fits perfectly. Great value for the money.

   sentiment
0      0.942
```

[114]:
```
reviews_clean.to_csv("reviews_clean.csv", index = False)
```

1.0.8 6. Product-Level Metrics

We combine ratings, sentiment, and review volume to understand product performance. This creates a single table with:

- Average rating
- Average sentiment
- Number of reviews
- Brand, category, price, image URL

[115]:
```
# combine review data with enriched product info (linked by SKU)
product_reviews = reviews_clean.merge(
    enhanced_products,
    on="sku",
    how="left"   # keep all reviews; attach product attributes when available
)

print("Merged Table:", product_reviews.shape)
#display(product_reviews.head())

# compute per-product aggregates: review stats + product metadata
product_metrics = (
    product_reviews
    .groupby("sku")
    .agg(
        avg_rating=("score", "mean"),            # average user rating
        avg_sentiment=("sentiment", "mean"),     # average sentiment score
        num_reviews=("score", "count"),          # review count
        name=("name", "first"),                  # basic product details
        description=("description", "first"),
        price=("price", "first"),
        brand=("brand", "first"),
        category=("category", "first"),
        category_tree=("category_tree", "first"),
        list_price=("list_price", "first"),
        sale_price=("sale_price", "first"),
        product_image_urls=("product_image_urls", "first")
    )
    .reset_index()
)

print("Product Metrics:", product_metrics.shape)
#display(product_metrics.head(2))
```

```
Merged Table: (63007, 17)
Product Metrics: (6043, 13)
```

1.0.9 7. Hidden Gem Score

This original metric highlights products that customers love but have low visibility.

Hidden Gem Score = (Rating x Sentiment)/ log(Number of Reviews)

High score = high potential product.

[116]:
```
import numpy as np

# Guard against missing values
product_metrics["avg_rating"] = product_metrics["avg_rating"].fillna(0)
product_metrics["avg_sentiment"] = product_metrics["avg_sentiment"].fillna(0)
product_metrics["num_reviews"] = product_metrics["num_reviews"].fillna(0)

# Compute the score
product_metrics["hidden_gem_score"] = (
    (product_metrics["avg_rating"] * product_metrics["avg_sentiment"]) /
    np.log(product_metrics["num_reviews"] + 2)
)

# Replace infinities with 0
product_metrics["hidden_gem_score"].replace([np.inf, -np.inf], 0, inplace=True)

# Sort from highest to lowest
hidden_gems = product_metrics.sort_values(
    by="hidden_gem_score",
    ascending=False
)

print("Top Hidden Gems:")
display(hidden_gems.head(2))
```

```
Top Hidden Gems:
```

	sku	avg_rating	avg_sentiment	num_reviews	\
3541	pp5006082043	5.000	0.988	1	
5513	pp5006750422	5.000	0.985	1	

	name	\
3541	ZCO Shape Bootcut Jeans - Plus	
5513	Carters® 3-pc. Seahorse Pajama Set - Toddler Girls 2t-5t	

	description	\
3541	Embellished back pockets give our bootcut jeans an eye-catching finish. elastic band provides comfort at the waist front mesh panels flatten the tummy area 5-pocket styling cotton/polyester/spande…	

```
5513  With choices of bottoms, this pajama set will make her feel extra comfy␣
 ↪every night. includes flutter-sleeve shirt, pants and shorts screen-print␣
 ↪graphics no-pinch elastic waistbands polyester was…

      price      brand category  \
3541 72.520  ZCO JEANS     sale
5513 31.420   Carter's  pajamas

                                       category_tree  list_price sale_price  \
3541  jcpenney|women|specialty-sizing|plus-size|sale      72.520      25.37
5513                           jcpenney|kids|pajamas      31.420       15.7

 ↪                                                                  ␣
 ↪                                                  product_image_urls  \
3541  http://s7d9.scene7.com/is/image/JCPenney/DP0824201517011894M.tif?
 ↪hei=380&amp;wid=380&op_usm=.4,.8,0,0&resmode=sharp2&op_usm=1.5,.
 ↪8,0,0&resmode=sharp
5513  http://s7d9.scene7.com/is/image/JCPenney/DP0216201617021841M.tif?
 ↪hei=380&amp;wid=380&op_usm=.4,.8,0,0&resmode=sharp2&op_usm=1.5,.
 ↪8,0,0&resmode=sharp

      hidden_gem_score
3541             4.495
5513             4.484
```

[117]:
```
# flag products whose hidden_gem_score is high enough to qualify as strong␣
 ↪hidden gems
threshold = 4.55 / 2   # 2.275

strong_hidden_gems = product_metrics[
    product_metrics["hidden_gem_score"] >= threshold
]
```

[118]:
```
strong_hidden_gems.to_csv("strong_hidden_gems.csv", index = False)
```

[119]:
```
percentage_strong = (len(strong_hidden_gems) / len(product_metrics)) * 100
percentage_strong
```

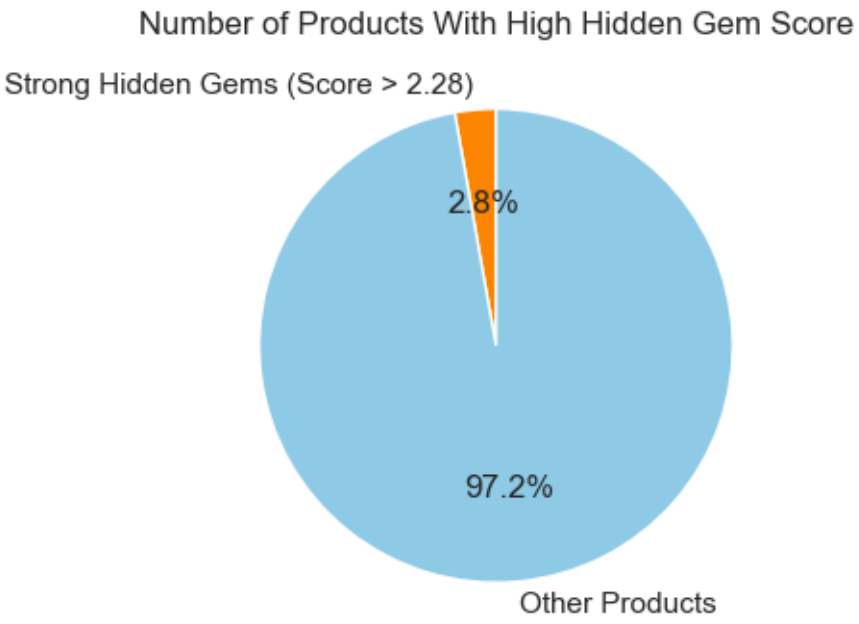
[119]:
```
2.796624193281483
```

1.0.10 8. Visualizing Product Performance

These charts highlight the hidden gems from whole data set. it shows that 2.8% of products are strong hidden gems.

[120]:
```
# pie chart comparing strong hidden gems vs. the rest of the catalog
labels = ["Strong Hidden Gems (Score > 2.28)", "Other Products"]
```

```
sizes = [len(strong_hidden_gems), len(product_metrics) -␣
 ↪len(strong_hidden_gems)]
colors = ["#fb8500", "#8ecae6"]

plt.figure(figsize=(4,4))
plt.pie(
    sizes,
    labels=labels,
    autopct="%1.1f%%",
    colors=colors,
    startangle=90
)
plt.title("Number of Products With High Hidden Gem Score")
plt.show()
```



[121]:
```
hg_metrics = pd.read_csv("strong_hidden_gems.csv")
```

Lets identify top three hidden gems

[122]:
```
# select the top 3 products by hidden gem score
top3 = (
    hg_metrics
    .sort_values("hidden_gem_score", ascending=False)
    .head(3)[["name", "avg_rating"]]
)
```

```
print("Top 3 Hidden Gem Products :")
display(top3)
```

```
Top 3 Hidden Gem Products :
```

	name	avg_rating
79	ZCO Shape Bootcut Jeans - Plus	5.000
143	Carters® 3-pc. Seahorse Pajama Set - Toddler Girls 2t-5t	5.000
1	Mens 6mm Tungsten Carbide Ring	5.000

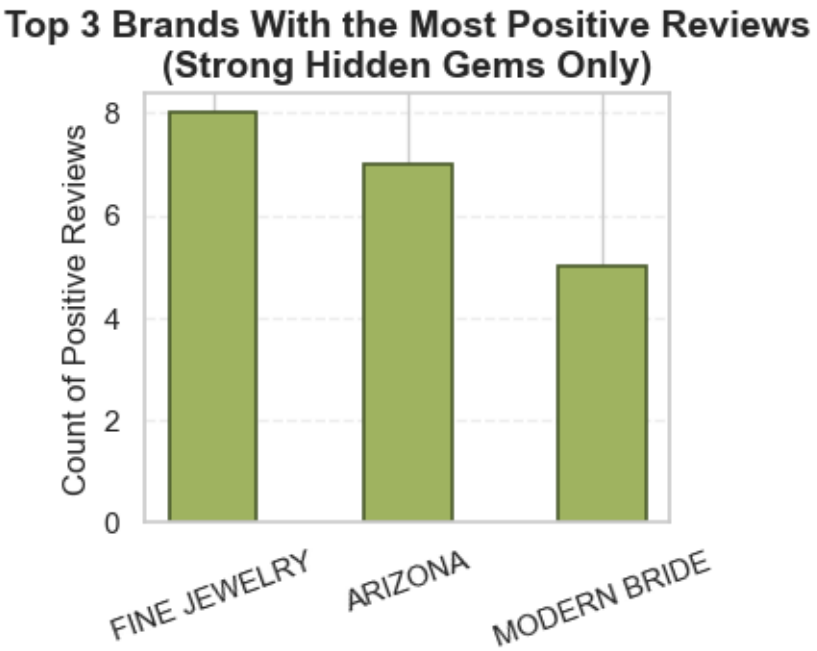
[123]:
```
positive = hg_metrics[hg_metrics["avg_sentiment"] > 0]
```

[130]:
```
import pandas as pd
import matplotlib.pyplot as plt
import requests
from PIL import Image
from io import BytesIO

# Plot setup
plt.figure(figsize=(15, 6))

for i, (_, row) in enumerate(top3.iterrows()):
    img_urls = str(row["product_image_urls"]).split(",")  # handle multiple URLs
    img_url = img_urls[0].strip()                          # take the first image

    # Fetch the image
    response = requests.get(img_url)
    img = Image.open(BytesIO(response.content))

    # Plot
    plt.subplot(1, 3, i+1)
    plt.imshow(img)
    plt.axis("off")
    plt.title(row["name"], fontsize=10)

plt.tight_layout()
plt.show()
```

We are going to identify top 3 brands as well

[131]:
```
top3_brands = (
    positive.groupby("brand")["avg_sentiment"]
    .count()
    .sort_values(ascending=False)
    .head(3)
)
```

[132]:
```
plt.figure(figsize=(4,4))

plt.bar(
    top3_brands.index,
    top3_brands.values,
    width=0.45,
    color="#9fb360",
    edgecolor="#4a5b2c",
    linewidth=1.2
)

plt.title("Top 3 Brands With the Most Positive Reviews\n(Strong Hidden Gems␣
 ↪Only)",
          fontsize=14, weight="bold")

plt.ylabel("Count of Positive Reviews", fontsize=12)

plt.xticks(rotation=20, ha="center", fontsize=11)
plt.yticks(fontsize=11)

plt.grid(axis="y", linestyle="--", alpha=0.35)

plt.tight_layout(pad=2)
plt.show()
```


Top 3 Brands With the Most Positive Reviews
(Strong Hidden Gems Only)
Count of Positive Reviews
0
2
4
6
8
FINE JEWELRY
ARIZONA
MODERN BRIDE